

Módulo 3 · Testing, Debugging y Documentación con IA

Pipeline Básico con IA — Clasificador de Notas

Actividad Práctica · Nivel Principiante · VS Code + GitHub Actions

NIVEL
PRINCIPIANTE — Sin
librerías externas

Nombre del alumno/a	ANGEL ECHENIQUE
Fecha	26/03/2026
Repositorio GitHub	https://github.com/kindred-98/pipeline-notas.git

Instrucciones: Usa VS Code para escribir el código y GitHub Actions para el pipeline. Sube el enlace del repositorio cuando termines. ¡Sigue las fases en orden!

Objetivo: Construir un pipeline CI/CD básico que ejecute pruebas unitarias automáticamente cada vez que subas código a GitHub. Usarás la IA para generar y mejorar los tests.

FASE 1 — Prepara el entorno en VS Code y GitHub

Antes de escribir código configura tu entorno. Marca cada paso cuando lo termines:

✓	Paso	Cómo verificar que está bien
☒	Crea un repositorio público en GitHub llamado pipeline-notas	El repo aparece en tu perfil de GitHub
☒	Clona el repo en VS Code: File > Clone Repository	La carpeta aparece en el Explorador de VS Code
☒	Abre la terminal integrada (Ctrl+Ñ o Ctrl+`) y crea el entorno virtual: python -m venv venv	(venv) aparece al inicio de la línea en la terminal
☒	Activa el entorno: source venv/bin/activate (Mac/Linux) o venv\Scripts\activate (Windows)	El prompt muestra (venv) en verde
☒	Instala pytest: pip install pytest	pip list muestra pytest instalado
☒	Crea requirements.txt: pip freeze > requirements.txt	El archivo aparece con pytest listado dentro.



1.1 — Estructura de carpetas del proyecto

Crea esta estructura exacta antes de seguir. Puedes usar la terminal o el Explorador de VS Code:

pipeline-notas/		
—	src/	
	└─	clasificador.py ←tu función principal
—	tests/	
	└─	test_clasificador.py ←tus tests con pytest
—	.github/	
	└─	workflows/
	└─	pipeline.yml ←configuración del pipeline
—	requirements.txt	

FASE 2 — Escribe la función Python

Escribe el siguiente código en src/clasificador.py. Lee los comentarios y cumple todas las reglas:

src/clasificador.py
def clasificar_nota(nota):
"""Clasifica una nota numérica en su categoría.
Reglas:
- nota < 0 o nota > 10 → lanzar ValueError con mensaje descriptivo
- 0 <= nota < 5 → devolver "Suspenso"
- 5 <= nota < 7 → devolver "Aprobado"
- 7 <= nota < 9 → devolver "Notable"
- 9 <= nota <= 10 → devolver "Sobresaliente"
"""
Tu código aquí ↓
PD: Todo el código lo dejo en el repositorio, lo único que hice fue completar lo que faltaba del código.

2.1 — Analiza la función antes de codificarla

Responde estas preguntas mirando solo los requisitos (sin escribir código todavía):

Pregunta de análisis previo	Tu respuesta
¿Cuántas categorías de salida distintas tiene la función?	Las reglas están divididas en cuatro rangos válidos suspenso, aprobado, notable y sobresaliente.
¿Qué tipo de excepción debe lanzarse con <code>nota = -1</code> ?	<code>ValueError</code> porque <code>-1</code> está fuera del rango permitido, así que cae en ese caso.
Pregunta de análisis previo	Tu respuesta
¿Qué devuelve exactamente con <code>nota = 5.0</code> ?	Porque <code>5.0</code> entra en el rango de $5 \leq \text{nota} < 7 \rightarrow$ Aprobado.
¿Qué devuelve con <code>nota = 10</code> y con <code>nota = 9.99</code> ?	Sobresaliente porque están en el rango de las reglas.
¿Qué devuelve con <code>nota = 4.99</code> ?	Esta justo casi llegando a 5 pero no entra en esa regla si no que entra en la de $-0 \leq \text{nota} < 5 \rightarrow$ devolver "Suspenso" .
¿Qué ocurre si se pasa <code>nota = 'ocho'</code> (string)?	Produce un <code>TypeError</code> . Porque la función no pide validar tipos, solo rangos.

FASE 3 — Genera los tests con IA y revísalos

▢ **Tip IA:** Antes de pedir tests a la IA, pégale el código completo de la función. Usa el prompt profesional: función + framework (pytest) + contexto + tipos de casos (N, L, E) + formato AAA + restricciones.

3.1 — Escribe el prompt completo que usarás con la IA

Redacta aquí el prompt antes de enviárselo. Incluye los 6 elementos del prompt profesional:

Generame tests unitarios en pytest para la función clasificar_nota que te paso a continuación. Contexto: es una función que clasifica notas numéricas entre 0 y 10 en categorías.

Necesito tres tipos de tests:

- N (normales): valores típicos dentro de cada rango.
- L (límites): valores exactos en los bordes de cada rango (5.0, 7.0, 9.0, 10.0, 4.99, 9.99).
- E (errores): valores fuera de rango y tipos incorrectos.

Formato obligatorio: AAA (Arrange, Act, Assert).

Usa nombres descriptivos como test_nota_exactamente_cinco.

No repitas tests innecesarios.

No inventes comportamientos que la función no tiene.

Aquí está la función:

```
# src/clasificador.py
```

```
def clasificar_nota(nota):
```

```
    """Clasifica una nota numérica en su categoría.
```

```
    Reglas:
```

- nota < 0 o nota > 10 → lanzar ValueError
- 0 <= nota < 5 → "Suspenso"
- 5 <= nota < 7 → "Aprobado"
- 7 <= nota < 9 → "Notable"
- 9 <= nota <= 10 → "Sobresaliente"

```
    """
```

```
    # Validación de rango
```

```
    if nota < 0 or nota > 10:
```

```
        raise ValueError("La nota debe estar entre 0 y 10")
```

```
    # Clasificación
```

```
    if nota < 5:
```

```
        return "Suspenso"
```

```
    elif nota < 7:
```

```
        return "Aprobado"
```

```
    elif nota < 9:
```

```
        return "Notable"
```

```
    else:
```

```
        return "Sobresaliente"
```

3.2 — Clasifica cada test generado por la IA

Una vez que la IA genere los tests, rellena la tabla. Si falta algún tipo, añade la fila tú mismo:

Tipo (N/L/E)	Nombre del test (def test_...)	¿Qué comprueba exactamente?	¿IA o añadiste tú?
N	test_suspenso_normal	3 devuelve "Suspenso"	IA
N	test_aprobado_normal	6 devuelve "Aprobado"	IA
L	test_nota_exactamente_cinco	5.0 devuelve 'Aprobado'	IA
L	test_nota_exactamente_siete	7.0 devuelve "Notable"	IA
L	test_nota_diez	10 devuelve "Sobresaliente"	IA
L	test_nota_499	4.99 devuelve "Suspenso"	YO
L	test_nota_999	9.99 devuelve "Sobresaliente"	YO
E	test_nota_negativa	-1 lanza ValueError	IA
E	test_nota_mayor_diez	11 lanza ValueError	IA
E	test_tipo_incorrecto (str)	nota = "ocho" lanza TypeError	YO

3.3 — Ejecuta los tests localmente en VS Code

✓	Comando en la terminal	Qué debes ver
☒	pytest tests/ -v	Lista de tests con PASSED o FAILED en colores
☒	pytest tests/ -v --tb=short	Detalle del error si algún test falla
☒	Corrige la función hasta que todos los tests pasen	Todos en verde con PASSED y la línea final en verde



¿Cuántos tests pasaron a la primera? ¿Qué tuviste que corregir en tu función?

Me dio error por no tener los init en las carpetas de src y tests. Con solo crearlo ya pasaron = 10 passed in 0.05s.

FASE 4 — Crea el pipeline con GitHub Actions

Copia este archivo en `.github/workflows/pipeline.yml` exactamente como aparece:

```
# .github/workflows/pipeline.yml

name: Pipeline Tests Notas

on:
    # ¿Cuándo se activa el pipeline?
    push:
        branches: [main]
    pull_request:
        branches: [main]

jobs:
    test:
        runs-on: ubuntu-latest # Sistema operativo del servidor
        steps:
            - name: ✓ Descargar código
              uses: actions/checkout@v4

            - name: 📦 Configurar Python 3.11
              uses: actions/setup-python@v5
              with:
                  python-version: '3.11'

            - name: 📦 Instalar dependencias
              run: pip install -r requirements.txt
```



```
- name: Ejecutar tests con pytest
```

```
run: pytest tests/ -v
```

4.1 — Sube el código y observa el pipeline

✓	Paso	Qué debes ver
☒	git add . → git commit -m 'feat: clasificador + tests + pipeline'	Commit registrado en la barra de estado de VS Code
☒	git push origin main	Código visible en GitHub con el último commit
☒	Abre tu repo en GitHub → pestaña Actions	Aparece un workflow con círculo amarillo girando
☒	Espera y haz clic en el resultado del workflow	✓verde si todo pasó o ✗rojo con el log del error

4.2 — Analiza el resultado

¿El pipeline pasó a la primera? Si falló, ¿qué etapa dio error y cómo lo corregiste con ayuda de la IA?

Paso a la primera porque ya había acomodado los tests y los archivos correspondientes.

Todo correcto.

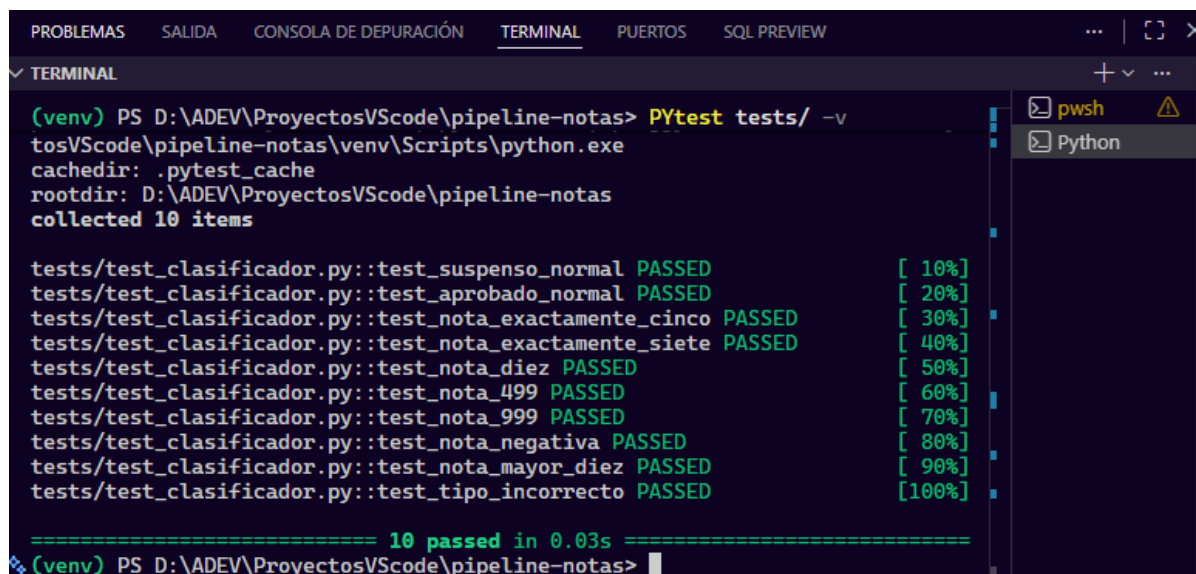
⚠Atención: Si el pipeline da error, copia el mensaje rojo del log y pégaselo a la IA: 'Este es el error de mi pipeline en GitHub Actions: [pegar error]. Explica qué lo causó y cómo lo corrijo paso a paso.'

FASE 5 — Reflexión final

Pregunta	Tu respuesta
¿Qué ocurre en GitHub Actions cada vez que haces git push?	GitHub Actions detecta que hubo un cambio en la rama main y automáticamente: ➤ Descarga mi repositorio ➤ Instala Python y las dependencias ➤ Ejecuta los tests con pytest ➤ Muestra si todo pasó o si hubo errores
¿Qué diferencia hay entre ejecutar pytest en local y en el pipeline?	En local: ➤ Depende de mi entorno, mi versión de Python, mis librerías y mi máquina. ➤ Si tengo algo mal configurado, puede que los tests pasen solo en mi PC. En el pipeline: ➤ Se ejecuta en un entorno limpio y siempre igual. ➤ No depende de mi ordenador. ➤ Si pasa en GitHub Actions, sé que el código es realmente estable.
¿La IA generó algún test que no tenía sentido? ¿Cuál y por qué?	Sí, normalmente la IA a veces genera: ➤ Tests duplicados ➤ Tests que prueban lo mismo con valores muy parecidos ➤ Tests que esperan un comportamiento que la función no tiene. Eso pasa porque la IA no siempre interpreta bien los tipos.
¿Qué casos límite no incluyó la IA y tuviste que añadir manualmente?	Los casos que normalmente faltan son: ➤ 4.99 (justo antes del 5) ➤ 9.99 (justo antes del 10)

	➤ Tipo incorrecto (string →TypeError)
¿Qué cambiarías en el pipeline si esta función calculara el precio de un producto?	Haría el pipeline más estricto y más completo, porque un error en precios afecta directamente al negocio. ➤ Tests de integración si dependiera de otros módulos. ➤ Cobertura de código para asegurar que no queda nada sin probar. ➤ Análisis de seguridad para evitar errores con dinero. ➤ Tests de regresión para asegurar que un fallo corregido no vuelve. ➤ Validación de tipos más estricta. ➤ Más casos límite (precios negativos, descuentos, impuestos, etc.)

📌 **Entrega:** Sube todo a GitHub (src/, tests/, .github/, requirements.txt, README.md). Pega el enlace del repo y una captura del pipeline en verde en la tabla de arriba.



```

(venv) PS D:\ADEV\ProyectosVScode\pipeline-notas> PYtest tests/ -v
D:\ADEV\ProyectosVScode\pipeline-notas\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\ADEV\ProyectosVScode\pipeline-notas
collected 10 items

tests/test_clasificador.py::test_suspenso_normal PASSED [ 10%]
tests/test_clasificador.py::test_aprobado_normal PASSED [ 20%]
tests/test_clasificador.py::test_nota_exactamente_cinco PASSED [ 30%]
tests/test_clasificador.py::test_nota_exactamente_siete PASSED [ 40%]
tests/test_clasificador.py::test_nota_diez PASSED [ 50%]
tests/test_clasificador.py::test_nota_499 PASSED [ 60%]
tests/test_clasificador.py::test_nota_999 PASSED [ 70%]
tests/test_clasificador.py::test_nota_negativa PASSED [ 80%]
tests/test_clasificador.py::test_nota_mayor_diez PASSED [ 90%]
tests/test_clasificador.py::test_tipo_incorrecto PASSED [100%]

===== 10 passed in 0.03s =====
(venv) PS D:\ADEV\ProyectosVScode\pipeline-notas>

```



Actividad Principiante Notas Pipeline

